PR6 – Programmation réseaux
Projet Paroles

Le but de ce projet est d'implémenter un serveur et un client IPv6 pour le protocole *Paroles* décrit ci-après.

I) Le protocole *Paroles*

Le protocole *Paroles* est un protocole de communication de type client/serveur. Des utilisateur·ices se connectent à un serveur avec un client et peuvent ensuite participer au fil de discussions des groupes auxquels iels sont abonné·es.
Plus précisément, un·e utilisateur·ice peut s'inscrire auprès du serveur et une fois inscrit·e, iel doit pouvoir faire les actions suivantes :

- créer un nouveau groupe et envoyer des invitations à des utilisateur·ices pour qu'iels rejoignent le groupe,
- recevoir des invitations pour s'abonner à des groupes existants,
- quitter un groupe,
- poster un nouveau billet sur le fil d'un groupe auquel iel appartient,
- répondre à un billet du fil d'un groupe auquel iel appartient,
- demander la liste des derniers billets non lus du fil d'un groupe auquel iel appartient,
- recevoir des notifications du serveur.

Le serveur doit répondre aux demandes du client et également envoyer des notifications.
Dans un premier temps, les échanges entre les clients et le serveur sont décrits de façon globale.
Le format exact des messages du client et du serveur est ensuite décrit précisément.

a) Les échanges client/serveur

L'inscription Lors de sa première connexion TCP au serveur, l'utilisateur·ice doit s'inscrire auprès du serveur. Les étapes des échanges entre le client et le serveur sont les suivantes :

1. le client se connecte en TCP au serveur et envoie une demande d'inscription en donnant le pseudo de l'utilisateur·ice, ainsi que sa clef publique (voir la section sécurité),
2. le serveur attribue alors un identifiant unique à l'utilisateur·ice, ainsi qu'un numéro de port pour la communication UDP et envoie ces informations au client avec, en plus, sa clef publique (voir la section sécurité). Il ajoute l'identifiant, le pseudo, la clef, l'adresse IP du client et le port UDP à la liste des inscrits.

   L'identifiant de l'utilisateur·ice l'identifie de façon unique. Deux utilisateur·ices peuvent avoir le même pseudo mais iels auront des identifiants distincts.
3. Le client récupère les informations envoyées par le serveur.
4. Le client et le serveur terminent la connexion.

Créer un nouveau groupe Lorsque l'utilisateur·ice souhaite créer un nouveau groupe, les étapes des échanges entre le client et le serveur sont les suivantes :

1. le client se connecte en TCP au serveur et envoie une demande de création d'un nouveau groupe en donnant le nom de ce groupe.

2. Le serveur attribue un identifiant unique strictement positif au nouveau groupe, ainsi qu'une adresse de multidiffusion unique et envoie ces informations au client. L'utilisateur·ice est l'administrateur·ice de ce nouveau groupe.

   L'identifiant du groupe l'identifie de façon unique. Deux groupes peuvent avoir le même nom mais ils auront des identifiants distincts.
3. Le client récupère l'identifiant et l'adresse de multidiffusion du groupe.
4. Le client et le serveur terminent la connexion.

Inviter des utilisateur·ices à rejoindre un groupe L'administrateur·ice d'un groupe peut demander au serveur d'envoyer des invitations à des utilisateur·ices pour qu'iels rejoignent le groupe. Les étapes des échanges entre le client et le serveur sont les suivantes :

1. le client se connecte en TCP au serveur et lui envoie un message contenant l'identifiant du groupe et les identifiants des utilisateur·ices que l'administrateur·ice souhaite inviter.
2. Le serveur répond par un acquittement au client et récupère le message.
3. Le client et le serveur terminent la connexion.

Demander les invitations en attente Un·e utilisateur·ice peut demander au serveur les invitations en attente qui lui sont adressées. Les étapes des échanges entre le client et le serveur sont les suivantes :

1. le client se connecte en TCP au serveur et lui envoie un message pour lui demander les invitations en attente.
2. Le serveur répond en envoyant pour chaque invitation, l'identifiant et le nom du groupe qu'il est invité à rejoindre, ainsi que le nom de l'administrateur·ice de ce groupe.
3. Le client récupère le message.
4. Le client et le serveur terminent la connexion.

Répondre à une invitation pour intégrer un nouveau groupe ou quitter un groupe Pour répondre à une invitation pour intégrer un nouveau groupe ou quitter un groupe, les étapes des échanges entre le client et le serveur sont les suivantes :

1. le client se connecte en TCP au serveur et envoie l'identifiant du groupe, soit qu'il est invité à rejoindre et s'il accepte ou refuse l'invitation, soit qu'il veut quitter.
2. Le serveur récupère le message et s'il correspond à une invitation acceptée, l'utilisateur·ice est ajouté·e au groupe. Dans ce cas, le serveur envoie au client un message contenant l'identifiant et le nom de l'administrateur·ice du groupe, l'adresse de multidiffusion du groupe et les identifiants et noms des utilisateur·ices appartenant à ce groupe.

   Si le message est une requête pour quitter le groupe, le serveur ôte l'utilisateur·ice de la liste des membres du groupe. Dans le cas où l'administrateur·ice demande à quitter le groupe, le serveur ferme définitivement le groupe qui disparaît du serveur.

   Le serveur répond par un acquittement au client si ce denier refuse une invitation ou s'il quitte un groupe.
3. Le client récupère ces informations.
4. Le client et le serveur terminent la connexion.

Demander la liste des membres d'un groupe ou de tous les utilisateur·ices L'utilisateur·ice peut rafraîchir sa liste des membres d'un groupe. Pour cela, les étapes des échanges entre le client et le serveur sont les suivantes :

1. le client se connecte en TCP au serveur et envoie l'identifiant du groupe dont il souhaite recevoir la liste des membres, ou il envoie l'identifant 0 s'il souhaite recevoir la liste de toustes les utilisateur·ices.
2. Le serveur récupère le message et envoie au client un message contenant le nombre de membres du groupe (respectivement d'utilisateur·ices inscrit·es), ainsi que l'identifiant et nom de chaque membre (respectivement utilisateur·ice inscrit·e). Si la requête concerne les membres d'un groupe, alors les premiers identifiant et nom correspondent à ceux de l'administrateur·ice du groupe.
3. Le client récupère ces informations.
4. Le client et le serveur terminent la connexion.

Poster un billet S'iel est inscrit·e, l'utilisateur·ice peut demander à poster un nouveau billet sur le fil d'un groupe auquel iel appartient. Les étapes des échanges entre le client et le serveur sont les suivantes :

1. le client se connecte en TCP au serveur et lui envoie un billet à ajouter en précisant l'identifiant du groupe sur le fil duquel il veut poster le billet.
2. Le serveur ajoute le billet au fil du groupe puis envoie un message au client avec le numéro du billet ajouté, ainsi que l'identifiant du groupe sur le fil duquel le billet a été ajouté.
3. Le client réceptionne la réponse du serveur.
4. Le client et le serveur terminent la connexion.

Répondre à un billet S'iel est inscrit·e, l'utilisateur·ice peut demander à répondre à un billet sur le fil d'un groupe auquel iel appartient. Les étapes des échanges entre le client et le serveur sont les suivantes :

1. le client se connecte en TCP au serveur et lui envoie son identifiant, le numéro du billet auquel il veut répondre, ainsi que l'identifiant du groupe auquel ce billet appartient, et sa réponse à ce billet.
2. Le serveur ajoute la réponse au fil du groupe puis envoie un message au client avec le numéro du billet, le numéro de la réponse ajoutée à ce billet, ainsi que l'identifiant du groupe sur le fil duquel la réponse été ajoutée.
3. Le client réceptionne la réponse du serveur.
4. Le client et le serveur terminent la connexion.

Demander la liste des derniers billets du fil d'un groupe S'iel est inscrit·e, l'utilisateur·ice peut demander tous les billets à partir d'un numéro de billet n et éventuellement d'un numéro de réponse à ce billet r sur le fil d'un groupe auquel iel appartient. Les étapes des échanges entre le client et le serveur sont les suivantes :

1. le client se connecte en TCP au serveur et lui envoie une demande de la liste des derniers billets postés sur le fil du groupe depuis le billet numéro n et et la réponse numéro r.
2. Le serveur envoie un message annonçant le nombre de billets m qu'il va envoyer au client et contenant également les m billets.

   Le serveur doit donc conserver l'ordre dans lequel il a reçu les billets et les réponses. Par exemple, si sur le fil d'un groupe donné :

  - Le serveur a reçu dans cet ordre, un billet b_1, un billet b_2, une réponse au billet b_2, un billet b_3, une réponse au billet b_1,
  - puis le serveur reçoit une requête demandant les billets depuis le billet b_2, il renvoie 3 billets : la réponse au billet b_2, le billet b_3 et la réponse au billet b_1,
  - puis le serveur reçoit une autre requête demandant les billets depuis la première réponse au billet b_2, il renvoie 2 billets : le billet b_3 et la réponse au billet b_1.
3. Le client réceptionne le message du serveur.
4. Le client et le serveur terminent la connexion.

La gestion des erreurs du client Le serveur ne doit pas répondre favorablement au client si :

- l'utilisateur·ice est inconnu·e (non inscrit·e),
- ou sa requête est mal formée,
- ou sa requête concerne un groupe auquel l'utilisateur·ice n'appartient pas,
- ou sa requête est une invitation pour rejoindre un groupe dont iel n'est pas l'administrateur·ice,
- ou s'il répond à une invitation alors que l'utilisateur·ice n'a pas été invité·e,
- ou sa requête concerne un groupe qui n'existe plus,
- ou tout autre requête inappropriée...

Les notifications Le serveur envoie des notifications à un groupe donné sur l'adresse de multidiffusion du groupe dans les cas suivants :

- lorsque le serveur reçoit un nouveau message à poster sur le fil du groupe,
- lorsqu'un utilisateur·ice accepte une invitation,
- lorsqu'un utilisateur·ice quitte le groupe,
- lorsque l'administrateur·ice ferme le groupe.

Le serveur envoie également des notifications à un·e utilisateur·ice donné·e, sur le port UDP determiné lors de la phase d'inscription, dans les cas suivants :

- lorsqu'il reçoit une invitation destinée à un·e utilisateur·ice, il envoie une notification d'invitation à cet·te utilisateur·ice,
- lorsqu'un utilisateur·ice récupère un billet, l'utilisateur·ice qui a écrit le billet est notifié·e,
- lorsqu'un utilisateur·ice envoie une réponse à un billet, l'utilisateur·ice qui a écrit le billet est notifié·e.

b) Les messages TCP du client et du serveur

Dans les messages :

- le champ `ID` est l'identifiant de l'utilisateur·ice,
- lorsqu'une ligne d'un message termine par l'acronyme `BE`, cela signifie que cette ligne forme un entier non signé au format big-endian,
- le champs `ZEROS` représente une suite de bits à zéro.

L'inscription Lors de sa première connexion au serveur, l'utilisateur·ice doit s'inscrire auprès du serveur en envoyant son pseudo. Le message d'inscription du client est donc

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |              ZEROS             |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                                                |
|                 NOM   10 octets                |
|                                                |
+------------------------------------------------+
|                                                |
|                 CLE   113 octets               |
|                                                |
+------------------------------------------------+
```

avec

- `CODEREQ = 1`,
- `NOM` est le pseudo de l'utilisateur·ice de 10 caractères maximum, avec complétion par des caractères nuls si le pseudo fait moins de 10 caractères,
- `CLE` correspond à la clef publique de l'utilisateur·ice sous forme d'une chaîne de caractères (voir la section sécurité pour plus d'informations).

La réponse du serveur est

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |               ID               |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                    PORTUDP                     |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                                                |
|                 CLE   113 octets               |
|                                                |
+------------------------------------------------+
```

avec

- `CODEREQ = 2`,
- `ID` est l'identifiant unique de l'utilisateur·ice attribué par le serveur,
- `PORTUDP` est le numéro de port sur lequel le client recevra les notifications UDP du serveur,
- `CLE` correspond à la clef publique du serveur sous forme d'une chaîne de caractères (voir la section sécurité pour plus d'informations).

Dans les messages du client, le champ `ID` correspond à l'identifiant unique de l'utilisateur·ice attribué par le serveur.

Créer un nouveau groupe Le client envoie le message

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |               ID               |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|          LEN          |       NOMG...
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 3`,

– `NOMG`, le nom du groupe avec `LEN` son nombre de caractères.

La réponse du serveur est

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                IDG             |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                PORTMDIFF                       |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                                                |
|                IPMDIFF   16 octets             |
|                                                |
+------------------------------------------------+
```

avec

– `CODEREQ = 4`,

– `IDG` est l'identifiant unique strictement positif du groupe attribué par le serveur,

– `PORTMDIFF` est le numéro de port de multidiffusion du groupe attribué par le serveur,

– `IPMDIFF` est un entier de 16 octets au format réseaux correspondant à l'adresse IPv6 de multidiffusion unique du groupe attribuée par le serveur.

Inviter des utilisateur·ices à rejoindre un groupe Le client envoie le message

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                ID              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|          IDG                   |     NB       |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|          ID1                   |     ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|          ID2                   |     ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                     ...
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

– `CODEREQ = 5`,

– `IDG` est l'identifiant du groupe,

– `NB` est le nombre d'invitations suivi des `NB` identifiants des utilisateur·ices invité·es et `ZEROS` (`ID1` et `ZEROS`, `ID2` et `ZEROS`...)

La réponse du serveur est

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |               ZEROS            |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec `CODEREQ = 24`.

Demander les invitations en attente Le client envoie le message

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                ID              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec `CODEREQ = 6`.

La réponse du serveur est

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                NB              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|               IDG1              |      LEN1    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                     NOMG1...
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                                                |
|                NOM1   10 octets                |
|                                                |
+------------------------------------------------+
|                  .....
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 7`,
- `NB` est le nombre d'invitations envoyées au client (qui peut valoir 0),
- puis pour chaque invitation sont envoyées, l'identifiant du groupe (`IDG`i), la longueur en octet du nom du groupe (`LEN`i), le nom du groupe (`NOMG`i) et le nom de l'administrateur·ice du groupe (`NOM`i).

Répondre à une invitation pour intégrer un nouveau groupe ou quitter un groupe
Le client envoie le message

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                ID              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|          IDG                    |  AN |  ZEROS |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 8`,
- `IDG` est l'identifiant du groupe,
- `AN` vaut 1 si une invitation est acceptée, 0 si elle refusée et 2 si l'utilisateur·ice demande à quitter le groupe.

Si `AN` vaut 1, la réponse du serveur est

```
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                IDG             |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                 PORTMDIFF                      |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                                                |
|                 IPMDIFF   16 octets            |   BE
|                                                |
+------------------------------------------------+
|                 NB                             |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|               ID1               |      ZEROS   |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                                                |
|                NOM1   10 octets                |
|                                                |
+------------------------------------------------+
|                  .....
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 9`,
- `IDG` est l'identifiant du groupe,
- `PORTMDIFF` est le numéro du port de multidiffusion du groupe,
- `IPMDIFF` est un entier de 16 octets au format réseaux correspondant à l'adresse IPv6 de multidiffusion,
- `NB` est le nombre de membres du groupe,
- puis pour chacun de ces membres sont envoyés, son identifiant (`ID`i) et son nom sur 10 octets (`NOM`i). Les premiers identifiant et nom doivent être celui de l'administrateur·ice du groupe.

Si `AN` vaut 0 ou 2, la réponse du serveur est

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |              ZEROS             |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec `CODEREQ = 24`.

Demander la liste des membres d'un groupe ou de tous les utilisateur·ices Si l'utilisateur·ice est membre d'un groupe, il peut demander la liste des membres de ce groupe ou de tous les utilisateur·ices. Pour cela, le client envoie le message

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                ID              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|          IDG                   |     ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 10`,
- `IDG` est l'identifiant du groupe si la requête concerne les membres du groupe, et vaut 0 si elle concerne toustes les utilisateur·ices inscrit·es.

La réponse du serveur est

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |               IDG              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                     NB                         |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|               ID1              |      ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                                                |
|                NOM1   10 octets                |
|                                                |
+------------------------------------------------+
|                  .....
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 11`,
- `IDG` est l'identifiant du groupe,
- `NB` est le nombre de membres du groupe,
- puis pour chacun de ces membre sont envoyés, son identifiant (`ID`i) et son nom sur 10 octets (`NOM`i). Les premiers identifiant et nom doivent être celui de l'administrateur·ice du groupe.

Poster un billet Le client envoie le message

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                ID              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|          IDG                   |     ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                      LEN                      |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                      DATA...
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 12`,
- `IDG` est l'identifiant du groupe,
- `LEN` est la longueur du billet en octets,
- `DATA` est le billet.

La réponse du serveur est

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |              IDG               |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                      NUMB      |     ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 13`,
- `IDG` est l'identifiant du groupe,
- `NUMB` est le numéro du billet attribué par le serveur. Les billets d'un groupe sont numérotées de façon incrémentale à partir de 0.

Répondre à un billet Le client envoie le message

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                ID              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|          IDG                   |     ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                      NUMB      |     ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                      LEN                      |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                      DATA...
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 14`,
- `IDG` est l'identifiant du groupe,
- `NUMB` est le numéro du billet auquel l'utilisateur·ice répond,
- `LEN` est la longueur de la réponse en octets,
- `DATA` est la réponse.

La réponse du serveur est

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |              IDG               |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                      NUMB      |     NUMR     |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 15`,
- `IDG` est l'identifiant du groupe,
- `NUMB` est le numéro du billet auquel l'utilisateur·ice répond,
- `NUMR` est le numéro de la réponse. Les réponses à un billet sont numérotées de façon incrémentale à partir de 1.

Demander la liste des derniers billets du fil d'un groupe Le client envoie le message

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |                ID              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|          IDG                   |     ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|               NUMB             |     NUMR     |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 16`,
- `IDG` est l'identifiant du groupe,
- `NUMB` est le numéro du dernier billet reçu avec le numéro de la dernière réponse à ce billet `NUMR` reçue.

La réponse du serveur est

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |               IDG              |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                     NB                         |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                ID1             |     ZEROS    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|               NUMB1            |     NUMR1    |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                     LEN1                       |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                     DATA1...
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                     .....
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 17`,
- `IDG` est l'identifiant du groupe concerné,
- `NB` est le nombre de billets qui vont être envoyés au client,
- puis pour chaque billet sont envoyées, l'identifiant de l'auteur du billet ou de la réponse (`ID`i), le numéro du billet (`NUMB`i), le numéro de la réponse (`NUMR`i) qui vaut zéro si ce n'est pas une réponse, la longueur en octets du billet ou de la réponse (`LEN`i) et le billet ou la réponse (`DATA`i).

La gestion des erreurs du client Lorsque le serveur ne peut répondre à la requête du client, car il a rencontré un problème (requête inappropriée ou mal formée), il envoie le message suivant au client :

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |             ZEROS             |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec `CODEREQ = 31`.
Extension possible du protocole (mais non obligatoire) : dans le message ci-dessus, on peut remplacer le champ `ZEROS` par la longueur de la chaîne de caractères qui suit et qui contient un message d'erreur. C'est pratique notamment pour debugger le code du serveur ou du client.

c) Les notifications du serveur

Toutes les notifications du serveur auront le format suivant :

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |               IDG             |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

correspondant à un entier encodé en big-endian avec `IDG`, l'identifiant du groupe concerné.
Le serveur envoie des notifications à un groupe donné sur l'adresse de multidiffusion du groupe dans les cas suivants :

- lorsque le serveur reçoit un nouveau message ou une réponse à poster sur le fil du groupe, il notifie les utilisateur·ices du groupe avec `CODEREQ = 18`,
- lorsqu'un·e utilisateur·ice accepte une invitation, le serveur notifie les membres du groupe avec `CODEREQ = 19`,
- lorsqu'un utilisateur·ice quitte le groupe, le serveur notifie les membres du groupe avec `CODEREQ = 20`,
- lorsque l'administrateur·ice ferme le groupe, le serveur notifie les membres du groupe avec `CODEREQ = 21`.

Le serveur envoie également des notifications à un·e utilisateur·ice donné·e en UDP dans les cas suivants :

- lorsqu'il reçoit une invitation destinée à un·e utilisateur·ice, il envoie une notification d'invitation à cet·te utilisateur·ice avec `CODEREQ = 22`,
- lorsqu'un utilisateur·ice récupère un billet, l'utilisateur·ice qui a écrit le billet est notifié·e avec `CODEREQ = 23`,

d) La sécurité

Afin de sécuriser les échanges entre le client et le serveur, les échanges TCP devront se faire avec le protocole TLS en utilisant la librairie `OpenSSL`.
Par ailleurs, pour éviter les problème d'usurpation d'identité, chaque connexion TCP, exceptée celle concernant l'inscription, devra être initiée par un premier message d'authentification du client, ainsi qu'une réponse d'authentification du serveur.
Le client et le serveur devront produire chacun leur paire de clés publique et privée de type ED25519 au format PEM. La longueur d'une clé publique ED25519 est de 60 octets. Si on ajoute les textes d'entêtes et de fin, le fichier contenant la clé a alors une taille de 113 octets.

C'est ces 113 octets qui sont inclus dans les messages de la phase d'inscription et correspondent au champ `CLE`.
Au début de chaque connexion TCP (exceptée celle concernant l'inscription), le client enverra un message contenant le code de la requête, son identifiant, un numéro et la signature avec sa clef privée de ces trois champs. Le serveur répondra par un message contenant ces mêmes trois champs et la signature avec sa clef privée de ceux-ci. Pour signer, vous devez utiliser la méthode montrée en cours. Les numéros débuteront à zéro et chaque nouvelle connexion avec authentification se fera avec le numéro incrémenté de 1.
Le message d'authentification pour le client et le serveur aura la forme suivante :

```
                                1
  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| CODEREQ      |              ID               |   BE
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
|                NUM  4 octets                  |
|                                               |
|--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--|
|        LSIG           |        SIG ...
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
```

avec

- `CODEREQ = 0`,
- `ID` est l'identifiant de l'utilisateur·ice,
- `NUM` est un entier non signé codé sur 4 octets au format big-endian correspondant au numéro du message d'authentification échangé,
- `LSIG` est un entier correspondant à la longueur de la signature `SIG`,
- `SIG` est la signature des trois premiers champs du message.

II) Le travail demandé

a) Réalisation

Le travail est à réaliser en trinôme. Il n'y aura pas de dérogation à cette règle sauf exception qui doit être argumentée. Vous devez envoyer par mail à `anne.micheli@irif.fr` au plus tard le 16 mars 2026, la composition de votre équipe avec les autres membres de l'équipe en copie du mail. L'objet du mail doit être **[PR6] équipe projet** et le corps du mail doit contenir 3 lignes, une par étudiant·e, contenant vos **Nom Prénom numéro-étudiant** (dans cet ordre).
Vous devez écrire en C le serveur et le client implémentant le protocole *Paroles*. Ils devront fonctionner sur IPv6.
Les erreurs devront être gérées, c'est-à-dire qu'il faut réfléchir aux actions en cas de réception d'un message mal formaté, d'un appel système erroné, d'une déconnexion non prévue du pair distant...
Il faudra également veiller à ce que vos applications ne bloquent pas éternellement. Pensez par exemple au cas du client qui initie une connexion TCP puis n'envoie jamais de requête. Cela signifie que vos applications doivent décider à un moment, qu'un pair n'est plus actif si cela fait trop longtemps qu'elles attendent un message.
La propreté du code, sa lisibilité seront également pris en compte. Pensez à commenter votre code et à un mode verbeux pour la soutenance. Evitez le code spaghetti, mutualisez le code... Et faites attention aux fuites de mémoire, de descripteurs...
Vous pouvez discuter entre vous du fonctionnement du protocole, du format des messages... mais vous ne pouvez pas vous échanger ou montrer du code. Tout code récupéré (sur internet, avec une IA générative...) doit le mentionner. Ce code ne sera pas pris en compte dans l'évaluation du projet. Le plagiat sans citer sa source est **strictement interdit**.

b) Organisation du travail

Le projet est conséquent. Il est donc conseillé de procéder par étape :

1. commencez par écrire une version simple du protocole, sans sécurité (sans signature et en utilisant TCP sans TLS). Cela signifie que les champs `CLE` lors de la phase d'inscription seront composés d'une suite de 113 caractères nuls et que l'échange préalable des messages d'authentification à chaque connexion TCP sera omis.
2. puis sécurisez les échanges TCP,
3. et à la fin, ajouter la gestion de l'authentification par signature.

c) Modalités de rendu

Dès que votre trinôme est constitué, vous devez créer un dépôt git **privé** sur le gitlab de l'UFR [1]. Tous les enseignants du cours devront y avoir accès en tant que **Reporter**, c'est-à-dire, Astyax Nourel, Fabien de Montgolfier, Peter Habermehl, Pierre Letouzey et Anne Micheli. Le dépôt devra contenir un fichier `authors.md` contenant la liste des membres de l'équipe (nom, prénom, numéro étudiant et pseudo(s) sur le gitlab).
Votre dépôt final devra contenir vos fichiers source ainsi qu'un `Makefile` pour la compilation. La **compilation et l'exécution devront fonctionner sur les machines de l'UFR**.
Ce rendu donnera lieu à une soutenance qui se déroulera en fin de période d'examens. Des précisions seront apportées ultérieurement.
Attention, les soumissions (commits) sur le gitlab rendront compte en partie de votre implication dans le projet. Si, par exemple, nous découvrons le jour de la soutenance en regardant sur le gitlab qu'une personne n'a pas participé au projet, sa note sera 0 au projet.

1. https ://moule.informatique.u-paris.fr/